Python Basics to ML: A Complete Journey

Training Report

Mac Tanatswa Muza | Registration No. 12405348

B.Tech. Computer Science and Engineering | Lovely Professional University

School of AI and Emerging Technologies | Centre for Professional Enhancement

Training Period: 11 June 2026 – 27 July 2026

Abstract— This report presents the work completed during the training program “Python Basics to ML: A Complete Journey.” The training began with core Python programming and gradually moved toward data handling and introductory machine learning. The practical work covered conditional and looping statements, strings, lists, tuples, functions, lambda functions, file handling, exception handling, and object-oriented programming. NumPy was introduced for numerical array operations, followed by basic data preprocessing through outlier detection using Z-score and interquartile range methods. The machine learning portion covered supervised and unsupervised learning, with practical exercises in Linear Regression, Logistic Regression, K-Nearest Neighbors, Naive Bayes, and K-Means clustering. A major practical component was customer churn prediction using the Telco Customer Churn dataset. The data was cleaned, categorical variables were encoded, features were standardized, and Logistic Regression, KNN, and Gaussian Naive Bayes were compared. Logistic Regression obtained the highest test accuracy of 81.55% in the implemented experiment. The training therefore provided a progression from basic programming to practical data analysis and machine learning, with emphasis on implementing concepts in Python rather than studying them only theoretically.

Keywords— Python, NumPy, Data Preprocessing, Machine Learning, Linear Regression, Classification, KNN, Naive Bayes, K-Means, Customer Churn

I. INTRODUCTION

The training program was designed as a progression from Python programming fundamentals to introductory machine learning. The supplied course document identifies the course as “Python Basics to ML: A Complete Journey” with Web ID 1349. The official project list describes the expected outcomes as developing machine learning skills, applying Python concepts, and analyzing and interpreting data. The practical files supplied for this report show that the training followed this general direction: programming fundamentals were practiced first, followed by numerical computing and data preparation, and then machine learning algorithms and a larger classification project.

The report is therefore written as a training report rather than as a single-project research paper. It records the main topics studied, the practical exercises completed, the tools used, and the major customer churn project. The purpose is to show the learning progression and the practical work performed during the training period.

II. TRAINING OBJECTIVES

- Build a working foundation in Python programming and problem solving.
- Use Python data structures, functions, file operations, and object-oriented programming in small practical programs.
- Develop basic numerical and data-handling skills using NumPy and Python-based data tools.
- Understand common machine learning problem types and the difference between supervised and unsupervised learning.
- Implement introductory regression, classification, and clustering algorithms using Python and scikit-learn.
- Practice basic data preprocessing and model evaluation.
- Apply the learned workflow to a larger customer churn classification project.

III. TRAINING CONTENT AND PROGRESSION

A. Python Programming Fundamentals

The first part of the training focused on writing and understanding basic Python programs. Exercises covered variables and data types, conditional statements, loops, match-case statements, strings, lists, tuples, and reusable functions. The string exercises included indexing, slicing, reversing, concatenation, membership checks, and common string methods. List exercises included adding, inserting, removing, and popping elements, while tuple exercises introduced immutable sequences, slicing, unpacking, reversing, and conversion to dictionaries.

Functions were practiced with parameters, return values, positional arguments, keyword arguments, default arguments, and variable-length arguments. Lambda functions were also used with map and filter operations. These exercises provided the basic programming skills needed later for data processing and machine learning code.

B. File Handling, Exceptions and Object-Oriented Programming

File handling exercises covered reading, writing, appending, line-by-line reading, and the use of the *with* statement. Exception handling covered try, except, else, and finally blocks, handling multiple exception types, catch-all handling, and raising a custom ValueError. These topics were useful for understanding how Python programs deal with files and runtime errors.

Object-oriented programming was practiced through classes and constructors, followed by inheritance, method overriding, method overloading approaches, abstraction, and data hiding. The supplied exercises included single, multiple, multilevel, and hierarchical inheritance. Abstract base classes were also used to demonstrate how a common interface can be defined for child classes.

C. NumPy and Numerical Operations

NumPy was introduced as a numerical computing library. The exercises created one-dimensional and two-dimensional arrays and performed array addition, sorting, indexing, slicing, summation, square roots, squares, and cumulative operations. Marks data was used for several operations, which made the examples closer to basic data-analysis tasks. This section helped connect normal Python programming with the array-based operations commonly used in machine learning workflows.

D. Data Preprocessing and Outlier Detection

The training also introduced the idea that data should be checked before it is used for analysis or modeling. An outlier was described as a value that is very different from the rest of the observations. The practical notebook demonstrated the Z-score method and the interquartile range (IQR) method. For Z-score detection, the notebook calculated the mean and standard deviation and selected observations with an absolute Z-score greater than 2. For the IQR method, Q1, Q3, the IQR, and lower and upper bounds were calculated before identifying values outside those bounds.

The exercise used a small series containing an unusually high value, allowing the effect of an outlier to be seen directly. This was an introductory treatment of preprocessing rather than a full data-cleaning pipeline.

IV. MACHINE LEARNING FUNDAMENTALS

The machine learning introduction covered the main categories of learning. Supervised learning was presented through classification and regression, while unsupervised learning included clustering. The material also introduced reinforcement learning concepts such as Q-Learning, SARSA, and Deep Q-Learning. Several algorithms were introduced at a conceptual level, including Logistic Regression, Decision Trees, KNN, Random Forest, SVM, Linear Regression, K-Means, DBSCAN, and Mean Shift.

This distinction between learning about an algorithm and implementing it was useful during the training. The exercises moved from simple datasets where the input and expected output were easy to understand to a larger customer dataset where preprocessing and model comparison became more important.

V. PRACTICAL MACHINE LEARNING EXERCISES

A. Linear Regression

Linear Regression was used to predict a continuous value from one or more input variables. The training notebook used a salary dataset containing YearsExperience and Salary and fitted a LinearRegression model. The slope, intercept, and a prediction for a new experience value were obtained from the model. Additional small examples related study hours to marks and plotted the observations together with a regression line. These exercises demonstrated the difference between an input feature and a continuous target and showed how a fitted model can be used for prediction.

Exercise	Input	Output
Salary prediction	YearsExperience	Salary
Study example	Hours	Marks

B. Logistic Regression

Logistic Regression was introduced as a classification method for a binary outcome. In the practical exercise, study hours were mapped to a Pass/Fail result. The model was trained using study hours as the feature and a binary result as the target, and then used to predict the result for a new number of study hours. A scatter plot was also created to visualize the relationship between study hours and the two outcome classes. This example provided a simple demonstration of classification before applying Logistic Regression to the larger churn dataset.

C. K-Nearest Neighbors

KNN was demonstrated using student marks in Mathematics and Computer Science. The training example contained five labeled observations and a new student with Maths = 6 and CS = 8. The exercise used K = 3 and asked the model to predict Pass or Fail. The notebook then implemented KNeighborsClassifier and plotted the observations together with the new student. The example helped connect the idea of distance between observations with the classification decision.

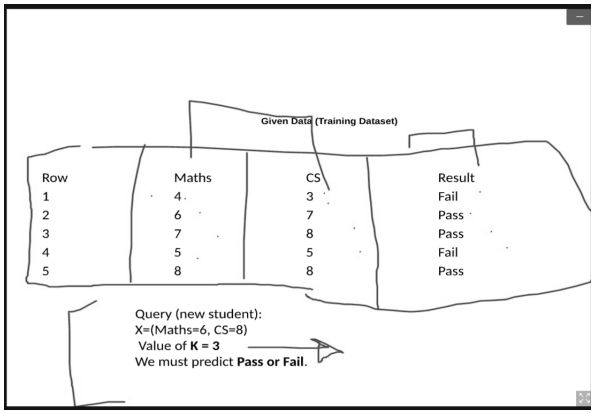


Fig. 1. KNN training example showing the labeled student data and a new query point.

D. Naive Bayes and Model Evaluation

Naive Bayes was introduced as a probability-based classification method. The weather/play exercise used categorical values, which were converted into numerical labels using LabelEncoder before training a CategoricalNB model. The model was then used to predict the class for a new weather value. A second part of the notebook introduced train-test splitting and evaluation using accuracy, confusion matrix, precision, recall, and F1-score. This was an important step because it showed that a model should be evaluated using measurable results rather than only checking whether individual predictions look correct.

E. K-Means Clustering

K-Means provided an introduction to unsupervised learning. The notebook used a small two-dimensional dataset and created a KMeans model with three clusters and random_state set to 0. The cluster labels and centroid coordinates were obtained after fitting the model, and a scatter plot was used to visualize the resulting groups and centroids. The exercise introduced the basic workflow of selecting K, assigning observations to clusters, and examining the resulting centroids.

VI. MAJOR PRACTICAL PROJECT: CUSTOMER CHURN PREDICTION

The larger practical project applied the training concepts to a real customer dataset. The project focused on predicting whether a telecommunications customer would churn. The supplied project material identifies Customer Churn Prediction as one of the training project options, alongside other Python and machine learning projects. This project was therefore used as the main demonstration of the complete workflow learned during the training.

A. Dataset and Preparation

The Telco Customer Churn dataset used in the notebook contains 7,043 customer records and 21 columns. The data includes customer information, service information, contract and billing information, with Churn used as the target. The preprocessing followed the order implemented in the notebook. Blank strings were replaced with missing values, TotalCharges was converted to numeric form, and missing TotalCharges values were filled using the column median. The customerID column was then removed because it is an identifier rather than a useful predictive characteristic.

Object-type columns were encoded with LabelEncoder. The target column Churn was separated from the feature matrix, and StandardScaler was applied to the feature matrix. The data was then divided into training and test sets using an 80/20 split with random_state = 42, producing 5,634 training rows and 1,409 test rows. One implementation limitation is that the scaler was fitted before the train-test split, so the scaling statistics were calculated using the full dataset. This is recorded here because the report describes the actual notebook implementation.

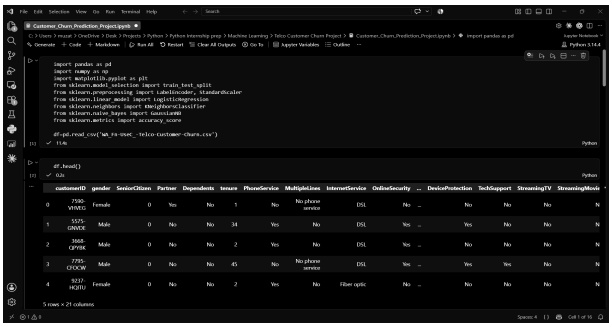


Fig. 2. Customer churn dataset loaded and inspected during the project.

B. Models Implemented

Logistic Regression: The notebook used LogisticRegression(max_iter=1000) for binary churn classification.

K-Nearest Neighbors: The notebook used KNeighborsClassifier(n_neighbors=5), so five neighboring observations were considered during prediction.

Gaussian Naive Bayes: The notebook used the default GaussianNB implementation and evaluated it on the same test set.

C. Results

All three models were evaluated using prediction accuracy on the held-out test set. Logistic Regression produced the highest test accuracy. The results were 81.55% for Logistic Regression, 75.73% for KNN, and 75.80% for Gaussian Naive Bayes. Thus, within this particular implementation and dataset, Logistic Regression performed approximately 5.82 percentage points better than KNN and 5.75 percentage points better than Gaussian Naive Bayes.

Model	Test Accuracy
Logistic Regression	81.55%
KNN	75.73%
Gaussian Naive Bayes	75.80%

TABLE I. CUSTOMER CHURN MODEL PERFORMANCE

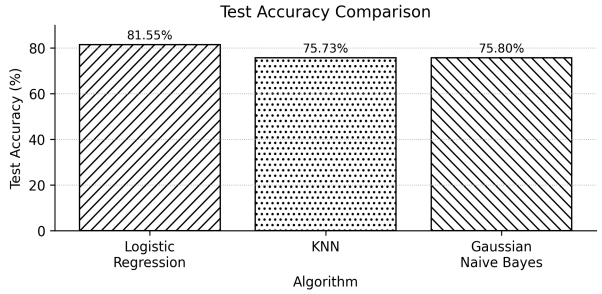


Fig. 3. Test accuracy comparison of the three customer churn models.

VII. TOOLS AND TECHNOLOGIES USED

The supplied practical files show regular use of Python and Jupyter Notebook-style workflows. NumPy was used for numerical arrays and operations, pandas was used for tabular data handling in the machine learning exercises, matplotlib was used for plots, and scikit-learn supplied the regression, classification, clustering, preprocessing, model-selection, and evaluation components used in the practical work. The training also used CSV files for datasets and ordinary Python files for several programming exercises.

VIII. SKILLS AND LEARNING OUTCOMES

The training developed a progression of practical skills rather than focusing on one isolated algorithm. At the programming level, the exercises strengthened Python syntax, functions, collections, file operations, exception handling, and object-oriented programming. At the data level, the work introduced NumPy arrays, basic preprocessing, and outlier identification. At the machine learning level, the training covered regression, classification, clustering, model evaluation, feature encoding, scaling, train-test splitting, and model comparison.

The customer churn project brought these ideas together in one workflow: loading data, inspecting it, cleaning selected fields, encoding categorical values, scaling features, splitting the dataset, training several classifiers, predicting the test labels, calculating accuracy, and comparing the models. This was the main practical outcome of the training because it required several concepts to be used together rather than in isolation.

IX. LIMITATIONS AND AREAS FOR IMPROVEMENT

The practical work was introductory and had several limitations. The customer churn project compared only three classification algorithms and used accuracy as its main reported result. The notebook did not report a complete set of churn-specific metrics for the final three-model comparison, and the hyperparameters were not systematically tuned. The categorical variables were encoded using LabelEncoder rather than a more detailed encoding strategy. The StandardScaler was also fitted before the train-test split, which means information from the test portion contributed to the scaling statistics. Finally, the project remained a notebook-based experiment and was not deployed as a production application.

Future work could include cross-validation, systematic hyperparameter tuning, confusion matrices and class-specific precision, recall and F1-score, ROC-AUC analysis, alternative categorical encoding, additional classification algorithms, and deployment of a trained model through a simple application. These are proposed improvements and were not treated as completed parts of the training.

X. CONCLUSION

The training program provided a practical path from Python programming fundamentals to introductory machine learning. The early exercises established programming skills, while NumPy and preprocessing exercises introduced the type of data operations used in machine learning. The later work implemented Linear Regression, Logistic Regression, KNN, Naive Bayes, and K-Means, giving practical exposure to both supervised and unsupervised learning. The customer churn project then combined these skills in a larger classification workflow and showed how different models can be compared on the same dataset.

The main learning outcome was not simply obtaining a high accuracy value from one model, but understanding the sequence from data preparation to model evaluation. In the completed churn experiment, Logistic Regression gave the highest test accuracy among the three tested models at 81.55%. This result is specific to the dataset and implementation used during the training, but it provided a useful example of how machine learning models can be implemented and compared in Python.

REFERENCES

[1] Lovely Professional University, “List of Summer Training/Internship Projects,” Python Basics to ML: A Complete Journey, Web ID 1349, 2026.

[2] Internal training notebooks supplied for the Python Basics to ML training program, including Python programming, NumPy, preprocessing, regression, classification, and clustering exercises, 2026.

[3] Internal project materials, “Customer Churn Prediction using Machine Learning,” supplied as part of the training work, 2026.

[4] scikit-learn developers, scikit-learn documentation and API references for LinearRegression, LogisticRegression, KNeighborsClassifier, Naive Bayes, KMeans, StandardScaler, and model evaluation utilities, accessed Aug. 20, 2026.

[5] NumPy Developers, NumPy documentation and API reference, accessed Aug. 20, 2026.